ECE1755 Assignment 1: Parallelizing 3D Ultrasound Beamforming
January 30, 2026

- Algorithm Analysis
The baseline beamforming algorithm reconstructs a 3D ultrasound image from transducer array data. It consists of two phases: (1) computing transmit distance from the transmitter to each image point, and (2) for each of the 32×32 transducer arrays, computing the receive distance to every image point, converting distance to an index, and accumulating the corresponding signal value. The key observation for parallelization is that each image point's computation is independent, which means its final value is simply the sum of contributions from all 1024 transducers with no dependencies on other points. This makes the algorithm parallel across the point dimension.
- Parallelization Approach: Pthreads + SIMD
I parallelized the algorithm at two levels. At the thread level, I partitioned the image point space across pthreads. Each thread receives a Task structure containing its assigned range (point_start, point_end) and pointers to shared read-only arrays (transducer positions, point coordinates, rx_data). Since threads write to non-overlapping regions of the output image array, no locks or synchronization are required. At the data level, I added SIMD vectorization for the performance competition. After checking CPU capabilities, I implemented AVX-512 vectorization (512-bit registers, 16 floats per operation). Key techniques include using _mm512_fmadd_ps for fused distance squared computation ($dx^2 + dy^2 + dz^2$), _mm512_i32gather_ps for vectorized scattered rx_data lookups, processing 4 transducers per iteration to maximize instruction-level parallelism.
- Performance and Scalability Results
Figure 1 (speedup_graph) shows speedup over baseline for all three input sizes across 1, 2, 4, 8, and 16 threads. For size-64, the AVX-512 implementation achieves: 7.1x (1 thread), 14.0x (2 threads), 27.4x (4 threads), 49.4x (8 threads), and 58.0x (16 threads). The speedup scales near-linearly from 1 to 8 threads—each doubling of threads roughly doubles performance. However, from 8 to 16 threads, speedup only increases from 49.4x to 58.0x (1.17x gain), indicating diminishing returns due to memory bandwidth saturation and hyperthreading overhead. Figure 2 (execution_time_size64) shows execution time versus thread count. The baseline takes 18.17s. With AVX-512 at 16 threads, execution drops to 0.31s. The graph also shows AVX-512 consistently outperforms AVX2 by approximately 2x at each thread count, reflecting the doubled vector width (512-bit vs 256-bit). At higher thread counts, this gap narrows slightly as memory bandwidth becomes the limiting factor rather than compute throughput.